Exercise 4.1 - Functions

Task 1 - Conversion of Units of Measurement

```
def convert_units(conversion_type, value):
    """
    Converts a value between different units of measurement.

    Parameters:
    conversion_type (int): The type of conversion selected by the user.
                           1 - Meters to Feet
                           2 - Feet to Meters
                           3 - Kilograms to Pounds
                           4 - Pounds to Kilograms
                           5 - Liters to Gallons
                           6 - Gallons to Liters
    value (float): The numeric value to be converted.

    Returns:
    float: The converted value.
    """

    if conversion_type == 1:  # Meters to Feet
        return value * 3.28084
    elif conversion_type == 2:  # Feet to Meters
        return value / 3.28084
    elif conversion_type == 3:  # Kilograms to Pounds
        return value * 2.20462
    elif conversion_type == 4:  # Pounds to Kilograms
        return value / 2.20462
    elif conversion_type == 5:  # Liters to Gallons
        return value * 0.264172
    elif conversion_type == 6:  # Gallons to Liters
        return value / 0.264172
    else:
        return None


def main():
    print("Unit Conversion Program")
    print("-----------------------")
    print("Length Conversions:")
    print("1. Meters (m) to Feet (ft)")
    print("2. Feet (ft) to Meters (m)")
    print("\nWeight Conversions:")
    print("3. Kilograms (kg) to Pounds (lbs)")
    print("4. Pounds (lbs) to Kilograms (kg)")
    print("\nVolume Conversions:")
    print("5. Liters (L) to Gallons (gal)")
    print("6. Gallons (gal) to Liters (L)")

    try:
        choice = int(input("\nEnter the conversion type (1-6): "))

        if choice not in range(1, 7):
            print("Error: Unsupported conversion type selected.")
            return

        value = float(input("Enter the value to convert: "))

        result = convert_units(choice, value)

        if result is not None:
            print(f"Converted value: {result:.4f}")
        else:
            print("Error: Conversion could not be performed.")

    except ValueError:
        print("Error: Please enter a valid numeric value.")
```

```
if __name__ == "__main__":
    main()
```

```
Unit Conversion Program
-----------------------
Length Conversions:
1. Meters (m) to Feet (ft)
2. Feet (ft) to Meters (m)

Weight Conversions:
3. Kilograms (kg) to Pounds (lbs)
4. Pounds (lbs) to Kilograms (kg)

Volume Conversions:
5. Liters (L) to Gallons (gal)
6. Gallons (gal) to Liters (L)

Enter the conversion type (1-6): 1
Enter the value to convert: 4.6
Converted value: 15.0919
```

Task 2 - Mathematical Operations

```
def calculate_sum(numbers):
    """
    Calculates the sum of a list of numbers.

    Parameters:
    numbers (list of float): The list of numbers to sum.

    Returns:
    float: The sum of the numbers.
    """
    return sum(numbers)


def calculate_average(numbers):
    """
    Calculates the average of a list of numbers.

    Parameters:
    numbers (list of float): The list of numbers to average.

    Returns:
    float: The average of the numbers.
    """
    return sum(numbers) / len(numbers)


def find_maximum(numbers):
    """
    Finds the maximum value in a list of numbers.

    Parameters:
    numbers (list of float): The list of numbers to evaluate.

    Returns:
    float: The maximum number in the list.
    """
    return max(numbers)


def find_minimum(numbers):
    """
    Finds the minimum value in a list of numbers.

    Parameters:
    numbers (list of float): The list of numbers to evaluate.

    Returns:
    float: The minimum number in the list.
    """
    return min(numbers)


def main():
    print("Mathematical Operations on a List of Numbers")
```

```
    print("-------------------------------------------")
    print("1. Sum")
    print("2. Average")
    print("3. Maximum")
    print("4. Minimum")

    try:
        choice = int(input("\nEnter the operation number (1-4): "))
        if choice not in [1, 2, 3, 4]:
            print("Error: Unsupported operation selected.")
            return

        input_str = input("Enter a list of numbers separated by spaces: ").strip()#removes leading and trailing whitespaces
        if not input_str:
            print("Error: The list cannot be empty.")
            return

        numbers = [float(x) for x in input_str.split()]#breaks a string into a list of substrings based on spaces

        if choice == 1:
            result = calculate_sum(numbers)
            print(f"Sum: {result}")
        elif choice == 2:
            result = calculate_average(numbers)
            print(f"Average: {result}")
        elif choice == 3:
            result = find_maximum(numbers)
            print(f"Maximum: {result}")
        elif choice == 4:
            result = find_minimum(numbers)
            print(f"Minimum: {result}")

    except ValueError:
        print("Error: Please enter valid numeric values.")


if __name__ == "__main__":
    main()
```

```
Mathematical Operations on a List of Numbers
-------------------------------------------
1. Sum
2. Average
3. Maximum
4. Minimum

Enter the operation number (1-4): 3
Enter a list of numbers separated by spaces: 1 8 3 4 9
Maximum: 9.0
```

Exercise 4.2 - List Manipulation

Task 1 - Extract Every Other Element

```
def extract_every_other(lst):
    """
    Extracts every other element from the input list, starting from the first element.

    Parameters:
    lst (list): The original list from which elements are to be extracted.

    Returns:
    list: A new list containing every other element from the original list.
    """
    return lst[::2]  # slice the list with a step of 2
    # [start:stop:step] -> default [0:end_of_list:1]

original_list = [1, 2, 3, 4, 5, 6]
result = extract_every_other(original_list)
print(result)
```

```
[1, 3, 5]
```

Task 2 - Slice a Sublist

```
def get_sublist(lst, start, end):
    """
    Returns a sublist from the input list, starting from index 'start' to index 'end' (inclusive).

    Parameters:
    lst (list): The original list.
    start (int): Starting index (inclusive).
    end (int): Ending index (inclusive).

    Returns:
    list: A sublist from index start to end.
    """
    # Python slicing is exclusive on the end index, so we add +1 to include it
    return lst[start:end+1]

original_list = [1, 2, 3, 4, 5, 6]
sublist = get_sublist(original_list, 2, 4)
print(sublist)
```

```
[3, 4, 5]
```

Task 3 - Reverse a List Using Slicing

```
def reverse_list(lst):
    """
    Reverses the input list using slicing.

    Parameters:
    lst (list): The original list to reverse.

    Returns:
    list: A new list that is the reverse of the input list.
    """
    return lst[::-1]  # slicing with step -1 reverses the list

original_list = [1, 2, 3, 4, 5]
reversed_list = reverse_list(original_list)
print(reversed_list)
```

```
[5, 4, 3, 2, 1]
```

Task 4 - Remove the First and Last Elements

```
def remove_first_last(lst):
    """
    Removes the first and last elements from a list using slicing.

    Parameters:
    lst (list): The original list.

    Returns:
    list: A new list without the first and last elements.
    """
    return lst[1:]  # slice from index 1 to the second-to-last element

original_list = [1, 2, 3, 4, 5]
result = remove_first_last(original_list)
print(result)
```

```
[2, 3, 4, 5]
```

Task 5 - Get the First n Elements

```
def get_first_n(lst, n):
    """
    Returns the first n elements from the input list using slicing.

    Parameters:
    lst (list): The original list.
    n (int): The number of elements to extract from the start.

    Returns:
```

```
    list: A new list containing the first n elements of the original list.
    """
    return lst[:n]  # slice from the start up to (but not including) index n

original_list = [1, 2, 3, 4, 5]
result = get_first_n(original_list, 3)
print(result)
```

```
[1, 2, 3]
```

Task 6 - Extract Elements from the End

```
def get_last_n(lst, n):
    """
    Returns the last n elements from the input list using slicing.

    Parameters:
    lst (list): The original list.
    n (int): The number of elements to extract from the end.

    Returns:
    list: A new list containing the last n elements of the original list.
    """
    return lst[-n:]  # slice from the nth element from the end to the last element

original_list = [1, 2, 3, 4, 5]
result = get_last_n(original_list, 2)
print(result)
```

Task 7 - Extract Elements in Reverse Order

```
def reverse_skip(lst):
    """
    Extracts every second element from the list in reverse order,
    starting from the second-to-last element.

    Parameters:
    lst (list): The original list.

    Returns:
    list: A new list containing every second element starting
          from the second-to-last element, moving backward.
    """
    return lst[-2::-2]  # start from second-to-last, step backwards by 2

original_list = [1, 2, 3, 4, 5, 6]
result = reverse_skip(original_list)
print(result)
```

Exercise 4.3 - Nested List

Task 1 - Flatten a Nested List

```
def flatten(lst):
    """
    Flattens a nested list into a single list (one-dimensional).

    Parameters:
    lst (list of lists): The original nested list.

    Returns:
    list: A flattened list containing all elements from the nested lists.
    """
    flat_list = []
    for sublist in lst:
        flat_list.extend(sublist)  # add elements of each sublist to flat_list
    return flat_list

nested_list = [[1, 2], [3, 4], [5]]
```

```
result = flatten(nested_list)
print(result)
```

```
[1, 2, 3, 4, 5]
```

Task 2 - Accessing Nested List Elements

```
def access_nested_element(lst, indices):
    """
    Accesses an element in a nested list using a list of indices.

    Parameters:
    lst (list): The nested list.
    indices (list of int): List of indices specifying the path to the element.

    Returns:
    The element at the specified position in the nested list.
    """
    element = lst
    for index in indices:
        element = element[index]  # go one level deeper for each index
    return element

nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
indices = [1, 2]
result = access_nested_element(nested_list, indices)
print(result)
```

```
6
```

Task 3 - Sum of All Elements in a Nested List

```
def sum_nested(lst):
    """
    Calculates the sum of all elements in a nested list (any depth).

    Parameters:
    lst (list): A nested list containing integers or other lists.

    Returns:
    int/float: The sum of all numbers in the nested list.
    """
    total = 0
    for item in lst:
        if isinstance(item, list): #checks whether the current item is a list
            # If the item is a list, call sum_nested recursively
            total += sum_nested(item)
        else:
            # If the item is a number, add it to the total
            total += item
    return total

nested_list = [[1, 2], [3, [4, 5]], 6]
result = sum_nested(nested_list)
print(result)
```

```
21
```

Task 4 - Remove Specific Element from a Nested List

```
def remove_element(lst, elem):
    """
    Removes all occurrences of a specific element from a nested list.

    Parameters:
    lst (list of lists): The original nested list.
    elem: The element to remove.

    Returns:
    list: A new nested list with all occurrences of elem removed.
    """
    new_list = []
    for sublist in lst:
```

```
        # Remove elem from each sublist using list comprehension
        new_sublist = [x for x in sublist if x != elem]
        new_list.append(new_sublist)
    return new_list

nested_list = [[1, 2], [3, 2], [4, 5]]
element_to_remove = 2
result = remove_element(nested_list, element_to_remove)
print(result)
```

```
[[1], [3], [4, 5]]
```

Text 5 - Find the Maximum Element in a Nested List

```
def find_max(lst):
    """
    Finds the maximum element in a nested list, regardless of depth.

    Parameters:
    lst (list): A nested list containing numbers or other lists.

    Returns:
    int/float: The maximum number in the nested list.
    """
    max_val = float('-inf')  # start with the smallest possible number
    for item in lst:
        if isinstance(item, list):
            # Recursively find the max in the sublist
            max_val = max(max_val, find_max(item))
        else:
            # Compare with current item
            max_val = max(max_val, item)
    return max_val

nested_list = [[1, 2], [3, [4, 5]], 6]
result = find_max(nested_list)
print(result)
```

```
6
```

Task 6 - Count Occurences of an Element in a Nested List

```
def count_occurrences(lst, elem):
    """
    Counts how many times 'elem' appears in a nested list 'lst'.
    """
    count = 0
    for item in lst:
        if isinstance(item, list):
            # Recursion: Ask the inner list how many times it found the element
            count += count_occurrences(item, elem)
        else:
            # Base Case: It's a single item, check if it matches
            if item == elem:
                count += 1
    return count

nested_list = [[1, 2], [2, 3], [2, 4]]
target = 2
result = count_occurrences(nested_list, target)
print(result)
```

```
3
```

Task 7 - Flatten a List of Lists of Lists

```
def deep_flatten(lst):
    """
    Flattens a deeply nested list into a single flat list.
    """
    flat_list = []
    for item in lst:
        if isinstance(item, list):
```

```
            # Recursion: Dive in and get the flat version of the sublist
            # Then extend our current list with those results
            flat_list.extend(deep_flatten(item))
        else:
            # Base Case: It's a single item, just add it to our list
            flat_list.append(item)
    return flat_list

nested = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
print(deep_flatten(nested))
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Task 8 - Nested List Average

```
def average_nested(lst):
    """
    Calculates the average of all elements in a nested list.

    Parameters:
    lst (list): A nested list containing numbers or other lists.

    Returns:
    float: The average of all numbers in the nested list.
    """

    def helper(sublist):
        total = 0
        count = 0

        for item in sublist:
            if isinstance(item, list):
                sub_total, sub_count = helper(item)  # recursive call
                total += sub_total
                count += sub_count
            else:
                total += item
                count += 1

        return total, count

    total_sum, total_count = helper(lst)
    return total_sum / total_count

nested_list = [[1, 2], [3, 4], [5, 6]]
result = average_nested(nested_list)
print(result)
```

```
3.5
```

Exercise 10.1 - Basic Vector and Matrix Operation with Numpy

Problem 1 - Array Creation

Task 1 - Initialize an empty array with size 2X2.

```
import numpy as np

array = np.empty((2,2))
print(array)
```

```
[[1.58421978e-315 0.00000000e+000]
 [4.94065646e-324 6.65817424e-310]]
```

Task 2 - Initialize an all one array with size 4X2.

```
import numpy as np
```

```
array = np.ones((4,2))
print(array)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

Task 3 - Return a new array of given shape and type, filled with fill value.

```
import numpy as np

array = np.full((3,2), 7)
print(array)
```

```
[[7 7]
 [7 7]
 [7 7]]
```

Task 4 - Return a new array of zeros with same shape and type as a given array.

```
import numpy as np

arr = np.full((3,2), 7)
zero_array = np.zeros_like(arr)
print(zero_array)
```

```
[[0 0]
 [0 0]
 [0 0]]
```

Task 5 - Return a new array of ones with same shape and type as a given array.

```
import numpy as np

arr = np.full((3,2), 7)
ones_array = np.ones_like(arr)
print(ones_array)
```

```
[[1 1]
 [1 1]
 [1 1]]
```

Task 6 - For an existing list new_list = [1,2,3,4] convert to an numpy array.

```
import numpy as np

new_list = [1,2,3,4]
np_array = np.array(new_list)
print(np_array)
```

```
[1 2 3 4]
```

Problem 2 - Array Manipulation: Numerical Ranges and Array Indexing

Task 1 - Create an array with values ranging from 10 to 49.

```
import numpy as np

array = np.arange(10, 50)
print(array)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
```

Task 2 - Create a 3X3 matrix with values ranging from 0 to 8.

```
import numpy as np

arr = np.arange(9)
array = arr.reshape((3,3))
print(array)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

Task 3 - Create a 3X3 identity matrix.

```
import numpy as np

arr = np.eye(3)
print(arr)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Task 4 - Create a random array of size 30 and find the mean of the array.

```
import numpy as np

arr = np.random.random(30) #values between 0 & 1
print(arr.mean())
```

```
0.4316176990569909
```

Task 5 - Create a 10X10 array with random values and find the minimum and maximum values.

```
import numpy as np

arr = np.random.random((10,10))
print(arr.min())
print(arr.max())
```

```
0.007537747581033871
0.9917376027035915
```

Task 6 - Create a zero array of size 10 and replace 5th element with 1.

```
import numpy as np

arr = np.zeros(10)
print("original array: ", arr)
arr[4] = 1
print("Modified array: ",arr)
```

```
original array:  [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
Modified array:  [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
```

Task 7 - Reverse an array arr = [1,2,0,0,4,0].

```
import numpy as np

arr = [1,2,0,0,4,0]
reversed_arr = arr[::-1]
print(reversed_arr)
```

```
[0, 4, 0, 0, 2, 1]
```

Task 8 - Create a 2d array with 1 on border and 0 inside.

```
import numpy as np

arr = np.zeros((5,5))
arr[0,:] = 1
arr[-1,:] = 1
```

```
arr[:, 0] = 1
arr[:, -1] = 1
print (arr)
```

```
[[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

Task 9 - Create a 8X8 matrix and fill it with a checkerboard pattern.

```
import numpy as np

arr = np.zeros((8,8))

arr[::2, 1::2]=1
arr[1::2, ::2]=1
print(arr)
```

```
[[0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]]
```

Problem 3 - Array Operations

Task 1 - Add the two array.

```
import numpy as np

x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11,12])

result_1 = np.add(x,y)
result_2 = np.add(v,w)

print("Adding x & y:\n", result_1)
print("Adding v & w:\n",result_2)
```

```
Adding x & y:
 [[ 6  8]
 [10 13]]
Adding v & w:
 [20 22]
```

Task 2 - Subtract the two array.

```
import numpy as np

x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11,12])

result_1 = np.subtract(x,y)
result_2 = np.subtract(v,w)

print("Subtracting y from x:\n",result_1)
print("Subtracting w from v:\n",result_2)
```

```
Subtracting y from x:
 [[-4 -4]
 [-4 -3]]
Subtracting w from v:
```

```
 [-2 -2]
```

Task 3 - Multiply the array with any integers of your choice.

```
import numpy as np

x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11,12])

result_1 = np.multiply(x,4)
result_2 = np.multiply(y,4)
result_3 = np.multiply(v,4)
result_4 = np.multiply(w,4)

print("Multiplying x with scalar 4:\n",result_1)
print("Multiplying y with scalar 4:\n",result_2)
print("Multiplying v with scalar 4:\n",result_3)
print("Multiplying w with scalar 4:\n",result_4)
```

```
Multiplying x with scalar 4:
 [[ 4  8]
 [12 20]]
Multiplying y with scalar 4:
 [[20 24]
 [28 32]]
Multiplying v with scalar 4:
 [36 40]
Multiplying w with scalar 4:
 [44 48]
```

Task 4 - Find the square of each element of the array.

```
import numpy as np

x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11,12])

result_1 = np.square(x)
result_2 = np.square(y)
result_3 = np.square(v)
result_4 = np.square(w)

print("Square of x:\n",result_1)
print("Square of y:\n",result_2)
print("Square of v:\n",result_3)
print("Square of w:\n",result_4)
```

```
Square of x:
 [[ 1  4]
 [ 9 25]]
Square of y:
 [[25 36]
 [49 64]]
Square of v:
 [ 81 100]
Square of w:
 [121 144]
```

Task 5 - Find the dot product between: v(and)w ; x(and)v ; x(and)y.

```
import numpy as np

x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11,12])

dot_vw = np.dot(v,w)
```

```
dot_xv = np.dot(x,v)
dot_xy = np.dot(x,y)

print("Dot product of v & w:\n",dot_vw)
print("Dot product of x & v:\n",dot_xv)
print("Dot product of x & y:\n",dot_xy)
```

```
Dot product of v & w:
 219
Dot product of x & v:
 [29 77]
Dot product of x & y:
 [[19 22]
 [50 58]]
```

Task 6 - Concatenate x(and)y along row and Concatenate v(and)w along column.

```
import numpy as np

x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11,12])

result_1 = np.concatenate((x,y), axis=0)

v_col = v.reshape(-1,1)
w_col = w.reshape(-1,1)

result_2 = np.concatenate((v_col, w_col), axis=1)

print("Conctatenating x & y along row:\n", result_1)
print("Conctatenating v & w along column:\n", result_2)
```

```
Conctatenating x & y along row:
 [[1 2]
 [3 5]
 [5 6]
 [7 8]]
Conctatenating v & w along column:
 [[ 9 11]
 [10 12]]
```

Task 7 - Concatenate x(and)v; if you get an error, observe and explain why did you get the error?

```
import numpy as np

x = np.array([[1,2],[3,5]])
v = np.array([9,10])

print(np.concatenate(x,v))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_198/618091793.py in <cell line: 0>()
      4 v = np.array([9,10])
      5
----> 6 print(np.concatenate(x,v))
      7
      8

TypeError: only integer scalar arrays can be converted to a scalar index
```

In the above question, we get an error because 'x' is a 2D array whereas 'v' is a 1D array & numpy cannot concatenate array with different dimensions.

Problem 4 - Matrix Operations

Task 1 - Prove A.A−1 = I.

```
import numpy as np

A = np.array([[3,4],[7,8]])
B = np.array([[5,3],[2,1]])

A_inv = np.linalg.inv(A)
I = np.dot(A, A_inv)

I_rounded = np.round(I, 6)
print(I_rounded)
```

```
[[1. 0.]
 [0. 1.]]
```

Task 2 - Prove AB≠ BA.

```
import numpy as np

A = np.array([[3,4],[7,8]])
B = np.array([[5,3],[2,1]])

AB = np.dot(A,B)
BA = np.dot(B,A)

print("Dot product of A & B is:\n",AB)
print("Dot product of B & A is:\n",BA)
```

```
Dot product of A & B is:
 [[23 13]
 [51 29]]
Dot product of B & A is:
 [[36 44]
 [13 16]]
```

Task 3 - Prove (AB)T = BTAT.

```
import numpy as np

A = np.array([[3,4],[7,8]])
B = np.array([[5,3],[2,1]])

AB = np.dot(A,B)
AB_T = AB.T

BTAT = np.dot(B.T, A.T)

print("(AB)T:\n", AB_T)
print("BTAT:\n", BTAT)
```

```
(AB)T:
 [[23 51]
 [13 29]]
BTAT:
 [[23 51]
 [13 29]]
```

Task 4 - Solve the following system of Linear equation using Inverse Methods.

2x − 3y + z = −1 x − y + 2z = −3 3x + y − z = 9

```
import numpy as np

A = np.array ([[2,-3,1], [1,-1,2], [3,1,-1]])
B = np.array([-1,-3,9])

A_inv = np.linalg.inv(A)

X = np.dot(A_inv, B)
print(X)
```

```
[ 2.  1. -2.]
```

Exercise 10.2 - Experiment: How Fast is Numpy?

Task 1 - Element-wise Addition

```
import numpy as np
import time

# Create two lists of size 1,000,000
size = 1_000_000
list1 = list(range(size))
list2 = list(range(size))

# Measure time
start_time = time.time()

# Element-wise addition using list comprehension
result_list = [list1[i] + list2[i] for i in range(size)]

end_time = time.time()
print("Python Lists: Time taken = {:.6f} seconds".format(end_time - start_time))
```

```
Python Lists: Time taken = 0.104826 seconds
```

```
import numpy as np
import time

# Create two NumPy arrays of size 1,000,000
array1 = np.arange(size)
array2 = np.arange(size)

# Measure time
start_time = time.time()

# Element-wise addition
result_array = np.add(array1, array2)

end_time = time.time()
print("NumPy Arrays: Time taken = {:.6f} seconds".format(end_time - start_time))
```

```
NumPy Arrays: Time taken = 0.005067 seconds
```

Task 2 - Element-wise Multiplication

```
import numpy as np
import time

# Create two lists of size 1,000,000
size = 1_000_000
list1 = list(range(size))
list2 = list(range(size))

# Measure time
start_time = time.time()

# Element-wise multiplication using list comprehension
result_list = [list1[i] * list2[i] for i in range(size)]

end_time = time.time()
print("Python Lists: Time taken = {:.6f} seconds".format(end_time - start_time))
```

```
Python Lists: Time taken = 0.222095 seconds
```

```
import numpy as np
import time

# Create two NumPy arrays of size 1,000,000
array1 = np.arange(size)
array2 = np.arange(size)

# Measure time
start_time = time.time()
```

```
# Element-wise multiplication
result_array = np.multiply(array1, array2)

end_time = time.time()
print("NumPy Arrays: Time taken = {:.6f} seconds".format(end_time - start_time))
```

```
NumPy Arrays: Time taken = 0.002951 seconds
```

Task 3 - Dot Product

```
import numpy as np
import time

# Size of the lists
size = 1_000_000

# Create two Python lists
list1 = list(range(size))
list2 = list(range(size))

# Measure time
start_time = time.time()

# Dot product using a loop / sum of element-wise products
dot_product_list = sum(list1[i] * list2[i] for i in range(size))

end_time = time.time()

print(f"Python Lists: Dot product = {dot_product_list}")
print(f"Python Lists: Time taken = {end_time - start_time:.6f} seconds")
```

```
Python Lists: Dot product = 333332833333500000
Python Lists: Time taken = 0.122859 seconds
```

```
import numpy as np
import time

# Create two NumPy arrays of size 1,000,000
array1 = np.arange(size)
array2 = np.arange(size)

# Measure time
start_time = time.time()

# Dot Product
result_array = np.dot(array1, array2)

end_time = time.time()
print(f"NumPy Arrays: Dot product = {result_array}")
print(f"NumPy Arrays: Time taken = {end_time - start_time:.6f} seconds")
```

```
NumPy Arrays: Dot product = 333332833333500000
NumPy Arrays: Time taken = 0.002528 seconds
```

Task 4 - Matrix Multiplication

```
import time
import random

# Size of matrices
n = 1000

# Create two 1000x1000 matrices using Python lists
A = [[random.random() for _ in range(n)] for _ in range(n)]
B = [[random.random() for _ in range(n)] for _ in range(n)]

# Initialize result matrix with zeros
result = [[0 for _ in range(n)] for _ in range(n)]

# Measure time
start_time = time.time()

# Matrix multiplication (triple nested loops)
for i in range(n):
```

```
    for j in range(n):
        for k in range(n):
            result[i][j] += A[i][k] * B[k][j]

end_time = time.time()
print(f"Python Lists: Time taken = {end_time - start_time:.6f} seconds")
```

```
Python Lists: Time taken = 337.328445 seconds
```

```
import numpy as np
import time

# Create two 1000x1000 matrices
A_np = np.random.rand(n, n)
B_np = np.random.rand(n, n)

# Measure time
start_time = time.time()

# Matrix multiplication using NumPy
result_np = np.dot(A_np, B_np)   # or A_np @ B_np

end_time = time.time()
print(f"NumPy Arrays: Time taken = {end_time - start_time:.6f} seconds")
```

```
NumPy Arrays: Time taken = 0.074453 seconds
```